Toward Accelerated LLM Inference: Port and Evaluate Diffusion-Based Speculative Decoding on TPU

Mentor: Hao Zhang — haz094@ucsd.edu
TA: Yiming Zhao — yiz279@ucsd.edu
Aaron Feng — zhf004@ucsd.edu
Zhongyan Luo — zhl203@ucsd.edu
Son Nguyen — hpn005@ucsd.edu
Andy Huang — anh031@ucsd.edu

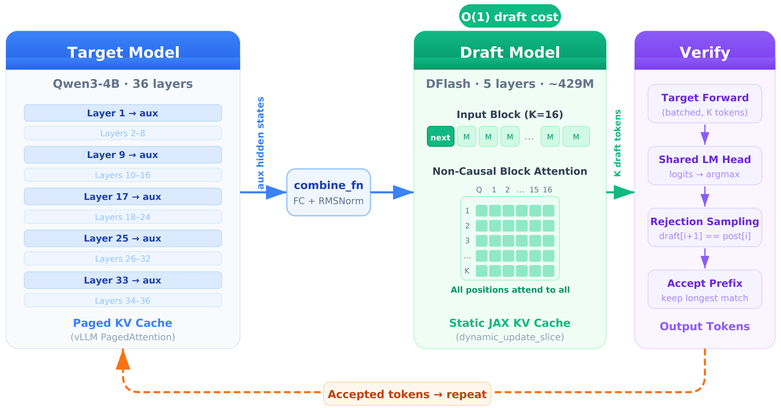
UC San Diego
HALICIOĞLU DATA SCIENCE INSTITUTE

Website GitHub

Introduction

Background

- **Autoregressive LLM decoding** is inherently sequential: generating n tokens requires n separate forward passes through the full model, making per-token latency the dominant **inference bottleneck**.
- **Speculative decoding** breaks this by introducing a lightweight draft model that proposes multiple candidate tokens at once. The target model **verifies all candidates in a single batched forward pass**, accepting a prefix of correct predictions.
- **DFlash** replaces the conventional autoregressive drafter with a **diffusion-based block drafter**: a compact 5-layer transformer that denoises 16 masked token positions in a single forward pass, producing all draft tokens in $O(1)$ **time** rather than Eagle3's $O(k)$.
- The DFlash draft model shares the target model's **embedding layer and LM head**, keeping parameter overhead minimal at $\sim$**429M parameters** trained on only **289K samples**.

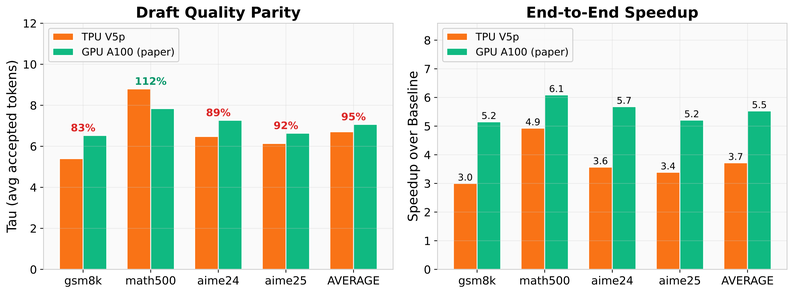


Motivation

- DFlash was developed exclusively for **GPU/PyTorch**, relying on mutable tensors and in-place cache operations. TPU requires **JAX**, a functional framework where all arrays are immutable, a fundamentally different programming model.
- **No prior work** has ported any diffusion-based speculative decoding method to TPU. The only existing TPU spec-decode implementation uses autoregressive drafting limited to $K{=}4$.
- Goal: port DFlash into **vLLM's TPU inference stack**, evaluate end-to-end viability across diverse benchmarks.
- Preview: TPU achieves **94.9%** of GPU draft quality (τ); MATH-500 **exceeds** GPU at **112%** ($\tau = 8.80$ vs. 7.84).

Draft Quality Parity

TPU V5p vs GPU A100 — Math Benchmarks



TPU V5P reaches **94.9% average τ parity** (6.71 vs. 7.07) across math benchmarks. MATH-500 **exceeds GPU** ($\tau = 8.80$ vs. 7.84). Remaining gap reflects **bf16 vs fp16** precision, not implementation errors. GPU reports higher speedup multipliers because its autoregressive baseline is slower.

Evaluation Setup

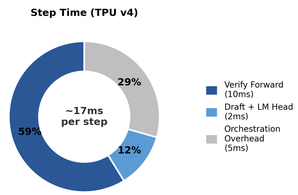
- **Hardware:** TPU V5P-8 node (4 chips, 2×2×1×1 topology). Standalone uses single device; vLLM uses 4-chip tensor parallelism.
- **Models:** Target: **Qwen3-4B** (36 layers, bf16). Draft: **DFlash-b16** (5 layers, 429M params, $K{=}16$). Shares target's embedding and LM head.
- **Benchmarks (9):** Math (GSM8K, MATH-500, AIME'24, AIME'25), Code (HumanEval, MBPP, SWE-Bench), Chat (MT-Bench, Alpaca).
- **Protocol:** Greedy decoding ($T{=}0$), 8 samples per dataset, max 256 new tokens. Timing with `jax.effects_barrier()` for device sync.

Methodology

Non-Causal Attention

- TPU runtime's default attention path (**ragged paged attention**) enforces **causal masking**, incompatible with DFlash's **bidirectional** draft block where all 16 positions attend to each other during denoising.
- Built `dflash_concat_attention`: concatenates context KV with noise block KV, applies **non-causal mask** within the draft block and causal mask to the prefix cache, routed through Pallas `flash_attention` with `causal=False`.
- This path **bypasses vLLM's paged attention entirely** for the draft model, running on a separate code path from Eagle3's autoregressive drafter.

KV Cache Design



- **Target model:** vLLM's standard paged KV cache. **Draft model:** pre-allocated contiguous JAX arrays updated immutably via `dynamic_update_slice`.
- **Three design iterations:** context-buffer-only (lost K/V history, $\tau{=}2.38$) $\rightarrow$ naïve per-layer cache (pytree shape mismatch triggered **JIT retracing**) $\rightarrow$ **pad to power-of-2** ($\sim$12 distinct shapes, retracing manageable).
- Step profiling: **59%** verify forward (10ms), **29%** orchestration overhead (5ms), **12%** draft+LM head (2ms). **Orchestration** is the primary optimization target.

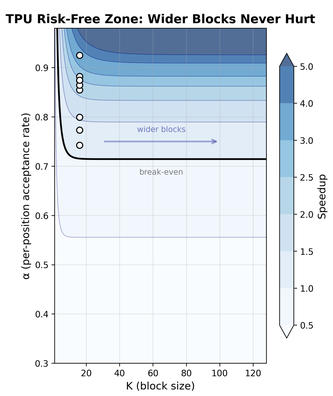
Key Optimization

- vLLM's speculative decoding manager passed `seq_lens` that included **unverified draft tokens**, silently inflating sequence length by **10–16 phantom tokens** per step.
- Corrupted **three subsystems simultaneously**: context buffer stored features for nonexistent positions, KV cache indices were offset, and **RoPE embeddings** accumulated 10–15 positions of drift per iteration.
- Bug was **non-crashing** and resided in upstream infrastructure code. Fix: **4 lines** extracting `num_tokens_no_spec` from the runner's input batch.

4-line fix: τ: 2.49 $\rightarrow$ **4.48** | Speedup: 1.30× $\rightarrow$ **2.31**× | Nearly **doubled** performance.

Verification

- Verification accounts for **59%** of step time (10ms/17ms), dominating throughput.
- TPU V5P: **1.31ms flat** from $K{=}16$ to $K{=}1024$ (ratio **1.00**×). Also flat across $L{=}64$–1024. Wider blocks add **zero overhead**.
- H100 also flat (**1.00–1.04**×). Only RTX 2000 Ada shows 1.24× (tiling artifact at $K{=}80$). K-flat is a **datacenter-class property**.
- **Why:** FFN is memory-bound. GPU attention scales 2.51× at $L{=}1024$, but TPU's RPA kernel absorbs it. Attention is only **2.1%** of total FLOPs.
- DFlash drafting also flat: **1.00**× on TPU, 1.02× on H100. Both sides are **K-invariant**.
- Throughput $= \tau(K)/[T_{\text{draft}} + T_{\text{verify}}]$. Both constant, so wider-block τ gains are **pure throughput**.
- Bottleneck: **draft quality** (α). At $\alpha{=}0.86$, $K{=}16$ captures 91% of $\tau(\infty)$. If $\alpha \rightarrow 0.90$, $K{=}64$ yields **53% more** tokens.



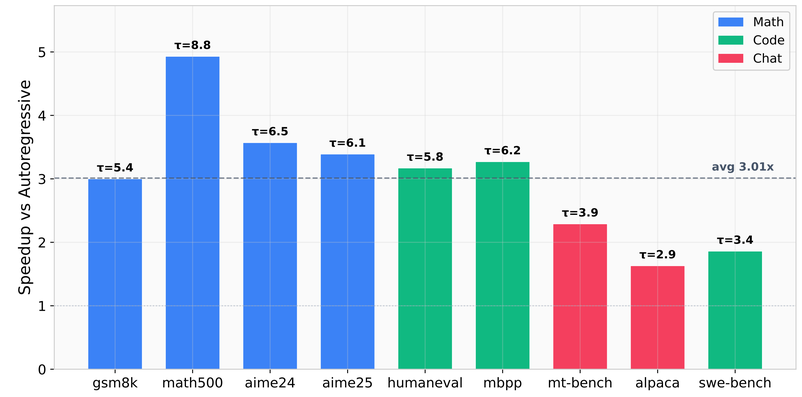
Acceptance Heatmap

Math benchmarks maintain >50% acceptance through positions 5–7; **code** drops around position 4–5; **chat** falls below 50% by position 2–3. MATH-500 sustains 0.23 at position 16; Alpaca reaches just 0.02. Validates **block size** $K{=}16$: at $\alpha \approx 0.86$, $K{=}16$ captures **91%** of theoretical $\tau(\infty)$.

Results

9-Benchmark Speedups

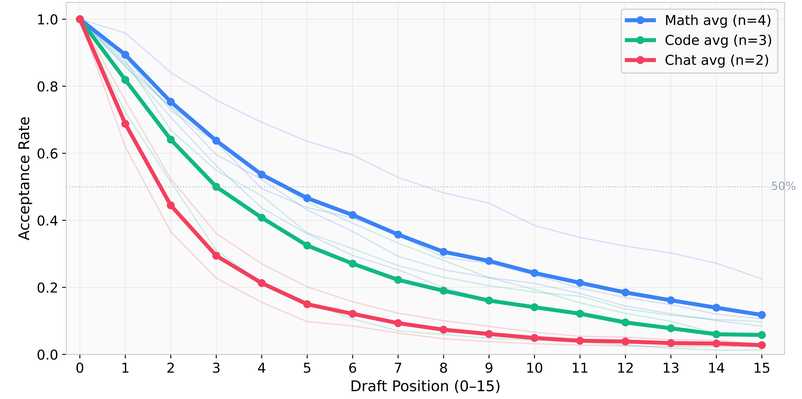
DFlash on TPU V5p — 9 Benchmarks



DFlash achieves **3.01**× **standalone speedup** overall, peaking at **4.93**× on MATH-500 ($\tau = 8.80$, 773 tok/s). The gradient **math 3.72**× > **code 2.77**× > **chat 1.96**× directly reflects decreasing token predictability. Even the lowest benchmark (Alpaca, 1.63×) provides meaningful speedup. Baseline TPOT is consistent at $\sim$6.3ms across all benchmarks, confirming speedup variation is driven by **draft quality** (τ), not hardware.

Acceptance Decay

Per-Position Acceptance by Category (TPU V5P)



Math (blue) sustains >50% acceptance through position 5–6; **code** (green) crosses 50% around position 3–4; **chat** (red) drops below 50% by position 2. The steeper the decay, the lower effective τ and speedup. At $\alpha \approx 0.86$ (math), $\tau(\infty) = 7.14$, so $K{=}16$ already captures 91% of the theoretical maximum.

Takeaways

- **DFlash transfers to TPU with near-full fidelity. 94.9%** of GPU draft quality, **3.01**× standalone speedup across 9 benchmarks. Port contributed upstream as a **3-PR chain to vllm-project/tpu-inference** (+2277 lines).
- **Systems debugging matters as much as algorithmic design.** A **4-line fix** to a silent sequence-length inflation bug nearly **doubled** pipeline performance (τ: 2.49 $\rightarrow$ 4.48).
- **Verification cost is flat on all datacenter hardware.** TPU V5P and H100 SXM show **no cost growth** from $K{=}16$ to $K{=}128$. Wider blocks are free; the bottleneck is **draft quality** (α).

Future Work

- **Pipeline optimization:** standalone 3.01× drops to 2.31× in vLLM. The LM head forces a **device-to-host sync** ($\sim$30% of step time); an on-device `jax.lax.while_loop` could eliminate this barrier.
- **Wider-block training** ($K{=}32$–128)**:** verification and drafting are both K-invariant. Open question: does wider bidirectional context improve α? DFlash $K{=}16$ beat $K{=}8$ by 21% on MATH-500.
- **Speculative Speculative Decoding (SSD):** DFlash's $O(1)$ drafting can fill **6**× **more cache entries** per verification window than autoregressive methods. Measured cache exponent: $r{=}0.81$ (math), $r{=}0.94$ (code).

Acknowledgements

We thank **Mentor Hao Zhang** and **TA Yiming Zhao** for guidance, UCSD HDSI and the DSC 180 Capstone program, and the **Google TPU Inference team** (Yarong Mu, Chengji Yao) for infrastructure collaboration.